AFRILANG Stdlib

std/c/cryptbase64

Français. Bibliothèque AFRILANG ‘std/c/cryptbase64‘ (niveau complex, catégorie complex). Elle expose 6 fonction(s) : encode, decode, isValid, paddedLen, roundTrip, urlSafe.
‘import ”std/c/cryptbase64”‘ · ‘use cryptbase64‘
- ‘encode(s text) → text‘ - ‘decode(s text) → text‘ - ‘isValid(s text) → bool‘ - ‘paddedLen(s text) → number‘ - ‘roundTrip(s text) → bool‘ - ‘urlSafe(s text) → text‘ **English.** AFRILANG library ‘std/c/cryptbase64‘ (tier complex, category complex). Exports 6 function(s): encode, decode, isValid, paddedLen, roundTrip, urlSafe.
‘import ”std/c/cryptbase64”‘ · ‘use cryptbase64‘
- ‘encode(s text) → text‘ - ‘decode(s text) → text‘ - ‘isValid(s text) → bool‘ - ‘paddedLen(s text) → number‘ - ‘roundTrip(s text) → bool‘ - ‘urlSafe(s text) → text‘

Catégorie / Category	Modules complex (c/)	
Niveau / Tier	complex	June 16, 2026
Fonctions / Functions	6	

Contents

Français

1. Vue d’ensemble

Bibliothèque AFRILANG ‘std/c/cryptbase64‘ (niveau complex, catégorie complex). Elle expose 6 fonction(s) : encode, decode, isValid, paddedLen, roundTrip, urlSafe.
‘import ”std/c/cryptbase64”‘ · ‘use cryptbase64‘

```
- `encode(s text) → text`
- `decode(s text) → text`
- `isValid(s text) → bool`
- `paddedLen(s text) → number`
- `roundTrip(s text) → bool`
- `urlSafe(s text) → text`
```

2. Installation & import

Ajoutez le module à votre programme :

```
import "std/c/cryptbase64"
use cryptbase64
```

```
Niveau : complex · Catégorie : Modules complex (c/)
Domaines avancés – graphes, NLP, réseaux complexes.
```

3. Référence API

```
•
 encode(s text) → text•
 decode(s text) → text•
 isValid(s text) → bool•
 paddedLen(s text) → number•
 roundTrip(s text) → bool•
 urlSafe(s text) → text
```

4. Exemples d’utilisation

```
import "std/c/cryptbase64"
use cryptbase64
```

```
create result = encode(/* args */)
say result
```

```
create result = decode(/* args */)
say result
```

```
create result = isValid(/* args */)
say result
```

```
create result = paddedLen(/* args */)
say result
```

```
create result = roundTrip(/* args */)
say result
```

```
create result = urlSafe(/* args */)
say result
```

5. Bonnes pratiques

```
•
 Préférez `import "std/c/cryptbase64"` en tête de fichier.•
 Lancez `afrilang check` avant de livrer.•
 Combinez avec les modules stdlib de la même catégorie.•
 Voir /stdlib/ pour le source live et les démos playground.•
 Signalez les problèmes sur GitHub si une export manque.
```

6. Annexe — source

module cryptbase64

```
function encode(s text) returns text
end
```

```
function decode(s text) returns text
end
```

```
function isValid(s text) returns bool
end
```

```
function paddedLen(s text) returns number
end
```

```
function roundTrip(s text) returns bool
end
```

```
function urlSafe(s text) returns text
end
end
```

English

1. Overview

AFRILANG library ‘std/c/cryptbase64‘ (tier complex, category complex). Exports 6 function(s): encode, decode, isValid, paddedLen, roundTrip, urlSafe.

‘import ”std/c/cryptbase64”‘ · ‘use cryptbase64‘

```
- `encode(s text) → text`
- `decode(s text) → text`
- `isValid(s text) → bool`
- `paddedLen(s text) → number`
- `roundTrip(s text) → bool`
- `urlSafe(s text) → text`
```

2. Installation & import

Add the module to your program:

```
import "std/c/cryptbase64"
use cryptbase64
```

```
Tier: complex · Category: Complex modules (c/)
Advanced domains — graphs, NLP, complex networks.
```

3. API reference

```
•
 encode(s text) → text•
 decode(s text) → text•
 isValid(s text) → bool•
 paddedLen(s text) → number•
 roundTrip(s text) → bool•
 urlSafe(s text) → text
```

4. Usage examples

```
import "std/c/cryptbase64"
use cryptbase64
```

```
create result = encode(/* args */)
say result
```

```
create result = decode(/* args */)
say result
```

```
create result = isValid(/* args */)
say result
```

```
create result = paddedLen(/* args */)
say result
```

```
create result = roundTrip(/* args */)
say result
```

```
create result = urlSafe(/* args */)
say result
```

5. Best practices

```
•
 Prefer `import "std/c/cryptbase64"` at the top of your file.•
 Run `afrilang check` before shipping.•
 Combine with related stdlib modules from the same category.•
 See /stdlib/ for the live source and playground demos.•
 Report issues on GitHub if an export is missing or undocumented.
```

6. Appendix — source

module cryptbase64

```
function encode(s text) returns text
end
```

```
function decode(s text) returns text
end
```

```
function isValid(s text) returns bool
end
```

```
function paddedLen(s text) returns number
end
```

```
function roundTrip(s text) returns bool
end
```

```
function urlSafe(s text) returns text
end
end
```

Référence rapide / Quick reference

- Import : `import "std/c/cryptbase64"`
- Vérification : `afrilang check`
- Documentation : https://afrilang.dev/stdlib/std/c/cryptbase64/

Source (extrait)

```
module cryptbase64

function encode(s text) returns text
end

function decode(s text) returns text
end

function isValid(s text) returns bool
end

function paddedLen(s text) returns number
end

function roundTrip(s text) returns bool
```

```
end

function urlSafe(s text) returns text
end
end
```